

REGULAR EXPRESSION



What is Java Regex

- **Java Regex, or Regular Expression**, is a powerful API used to **define search patterns** for matching, searching, and manipulating strings.
It allows you to express complex string conditions using concise patterns.
- Regular expressions are widely used to enforce constraints on input, for example:
- **Password validation**
(e.g., must contain uppercase, lowercase, digits, special characters)
- **Email validation**
- **Phone number formats**
- **Date formats**
- **Extracting specific parts of text**

Searching for things

- When you want to search for a file in Windows, you can specify wildcards in the filename...

Pattern	Matches...
Test.txt	Only that filename
Test.*	All files called Test with any extension
Test*.txt	Example: Test1.txt, Test2.txt
.	Any file

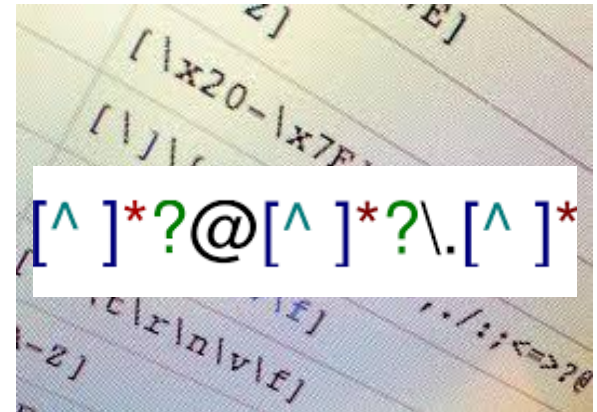


Wildcard Searching

- This isn't quite regular expressions, but it's similar
- The * character is a meta-character, which means it isn't interpreted literally, but has a special meaning instead
- In the case of Windows file searches, we can use the following meta-characters:
 - * : 0 or more occurrences of any character
 - ? : 1 occurrence of any character

Regular Expressions

- These are a much more powerful set of syntax rules for searching and replacing within strings
- There are different dialects of regular expression syntax: Perl, grep, Posix, etc
- The regular expression syntax used by Java is very similar to the syntax used in Perl



Java's Regex Classes

- All in the `java.util.regex` package
- `Pattern` – this is regular expression itself which is compiled
- `Matcher` – this is engine which matches the regular expression against an input string
- `PatternSyntaxException` – what you'll get if you try to create a `Pattern` from an invalid regular expression



The Pattern

- We can't create a Pattern object directly because it has no public constructor
- We use the static `compile` method instead, e.g.

```
Pattern pattern1 = Pattern.compile("foo");  
Pattern pattern2 = Pattern.compile("foo.*");  
Pattern pattern3 = Pattern.compile("f.o?");
```

The Matcher

- We don't create the matcher directly either
- We use the `matcher` method of the pattern object to create a `Matcher` from the string we want to search in

```
Matcher matcher = pattern.matcher("foofoofoo");
```



Getting the results

- The `find` method of the `Matcher` class will return `true` if it can match the pattern to the input string, e.g.

Pattern	Input	<code>find()</code>
<code>cat</code>	<code>cat</code>	<code>true</code>
<code>cat</code>	<code>cats and dogs</code>	<code>true</code>
<code>cats</code>	<code>my cat</code>	<code>false</code>
<code>z</code>	<code>my cat</code>	<code>false</code>

Getting the results

- If `find` returns true, we can get information from the matcher about what part of the input string matched the pattern
- The matcher class gives us these methods...
 - `group()` : returns the substring of the input string that matched the pattern
 - `start()` : returns the starting index of that substring (inclusive)
 - `end()` : returns the end index of that substring (exclusive)

Getting the results

- The pattern might be found several times within the input string, so if we want all matches, we need to loop...

```
Matcher matcher = pattern.matcher("foofoofoo");  
while (matcher.find()) {  
    String group = matcher.group();  
    int start = matcher.start();  
    int end = matcher.end();  
  
    // TODO Process this result!  
}
```

Example: Getting all matches

- The pattern might be found several times within the input string, so if we want all matches, we need to loop...

```
Pattern pattern = Pattern.compile("foo");  
Matcher matcher =  
pattern.matcher("foofoofoo");  
while (matcher.find()) {  
    String group = matcher.group();  
    int start = matcher.start();  
    int end = matcher.end();  
}
```

group()	start()	end()
foo	0	3
foo	3	6
foo	6	9

Matching with OR (|)

- We can let the matcher matches a string or character from a set of options using "|"

Pattern	Input	find()
cat dog	cat	true
cat dog	the man's dog	true
0 1 2	1	true

Meta-character: the period (.)

- A period is a *meta-character* in regex
- It matches ANY character

Pattern	Input	find()
c.t	cat	true
c.t	cut	true
c..t	cat	false
..t.	cats	true

Escaping meta-characters

- If we want the matcher to treat a period (or any other meta-character) as a literal character, we need to escape it
 - We can prefix it with a backslash, e.g. `\.`
 - We can put it between `\Q` and `\E`, e.g. `\Q...\E`

Pattern	Input	<code>find()</code>
<code>ca.</code>	<code>cat</code>	<code>true</code>
<code>ca\.</code>	<code>cat</code>	<code>false</code>
<code>ca\.</code>	<code>ca.</code>	<code>true</code>
<code>\Q...\E</code>	<code>cats...</code>	<code>true</code>

Character Classes

- We can specify a set of characters to be matched within a string
- The set is placed inside square brackets, e.g. `[abc]` matches "a", "b" or "c",
- Ranges can also be defined using "-", e.g. `[a-z]` matches any character between "a" and "z" (inclusive)
- The class can be negated using "^", e.g. `[^abc]` matches any character except "a", "b" or "c"

Simple Character Class Examples

Pattern	Will match...
<code>c[au]t</code>	cat and cut but not cot
<code>c[^au]t</code>	cot but not cat or cut
<code>c[A-Z]t</code>	cUt but not cut
<code>c[a-zA-Z]t</code>	cat and cAt
<code>TEST[0-4]</code>	TEST1 and TEST4 but not TEST5
<code>TEST[\\.\\]</code>	TEST. and TEST\ but not TESTX
<code>TEST[^\\.a-z]</code>	TEST1 but not TEST. or TESTz

Advanced Character Classes

- By nesting one character range inside another, we can define a union, e.g.
 - `[a-d[m-p]]` – matches `a-d` or `m-p`
- By using `&&` we can define an intersection of two ranges, e.g.
 - `[a-f&&[d-h]]` – matches `d`, `e` and `f`
- Combining that with a `^` negation, we can subtract items from a range, e.g.
 - `[a-z&&[^bc]]` – matches `a-z` except `b` or `c`
 - `[a-z&&[^m-p]]` – matches `a-z` except `m-p`

Advanced Character Classes

Pattern	Will match...
<code>[0-2[6-9]]</code>	only 0, 1, 2, 6, 7, 8 or 9
<code>[1-5&&[3-8]]</code>	only 3, 4 or 5
<code>[0-9&&[^3-5]]</code>	only 0, 1, 2, 6, 7, 8 or 9
<code>c[a-z&&[^o]]t</code>	cat and cut but not cot
<code>[0-9][0-9]</code>	00, 01, 02, 03 ... 97, 98 or 99

Predefine Character Classes

- There are several predefined character classes for the sake of convenience...

Class	Equivalent to...
<code>\d</code>	Any digit: <code>[0-9]</code>
<code>\D</code>	Any non-digit: <code>[^0-9]</code>
<code>\s</code>	A whitespace character: <code>[\t\n\x0B\f\r]</code>
<code>\S</code>	A non-whitespace character: <code>[^\s]</code>
<code>\w</code>	A word character: <code>[a-zA-Z_0-9]</code>
<code>\W</code>	A non-word character: <code>[^\w]</code>

Predefined Class Examples

- If you want to write these as string literals in your code, then the backslashes must be escaped, e.g. `\d` → `\\d`

Pattern	Will match...
<code>cat\d</code>	<code>cat1</code> or <code>cat2</code> but not <code>catx</code>
<code>my\scat</code>	<code>my cat</code> but not <code>mycat</code>
<code>c\wt</code>	<code>cat</code> or <code>c_t</code> but not <code>c-t</code>

Quantifiers

- These allow you to specify the number of occurrences of something to match
- The simplest are the "greedy" quantifiers...

$X?$	Matches X once or not at all
X^*	Matches X zero or more times
X^+	Matches X one or more times
$X\{n\}$	Matches X exactly n times
$X\{n, \}$	Matches X at least n times
$X\{n, m\}$	Matches X at least n times but no more than m times

Classes with Quantifiers

- Normally the quantifier only applies to the last character, i.e. `abc*` matches `abccc`
- But we can apply quantifiers to classes, e.g.
 - `[a-z]*` - means zero or more of the characters `a` to `z`
 - `[012]+` - means one or more of the characters `0`, `1` and `2`

Groups with Quantifiers

- Groups can be defined by using parenthesis
- Quantifiers can then be applied to those groups, e.g.
 - `(cat) *` - matches `dogcatcatcatdog`
 - `(123) {2}` - matches `32123123`

Greedy Quantifier Examples

Pattern	Will match...
<code>ca?t</code>	<code>cat</code> or <code>ct</code> but not <code>caat</code>
<code>ca*t</code>	<code>ct</code> , <code>cat</code> or <code>caaaat</code>
<code>ca+t</code>	<code>cat</code> , <code>caat</code> but not <code>ct</code>
<code>jb0{2}7</code>	<code>jb007</code> but not <code>jb07</code> or <code>jb0007</code>
<code>t.*n</code>	<code>ton</code> , <code>town</code> , <code>tan</code> or <code>then</code>
<code>cat!?</code>	<code>cat!</code> or <code>cat</code>

Reluctant Quantifiers

- Greedy quantifiers will match as much of the input as they can by working from the end
- Reluctant quantifiers only return the shortest next bit of the input that matches
- To make a quantifier reluctant, put an extra `?` on the end of it, e.g.
 - `X?` becomes `X??`
 - `. *` becomes `. *?`
 - `[abc]{3}` becomes `[abc]{3}?`

Example: Reluctant vs. Greedy

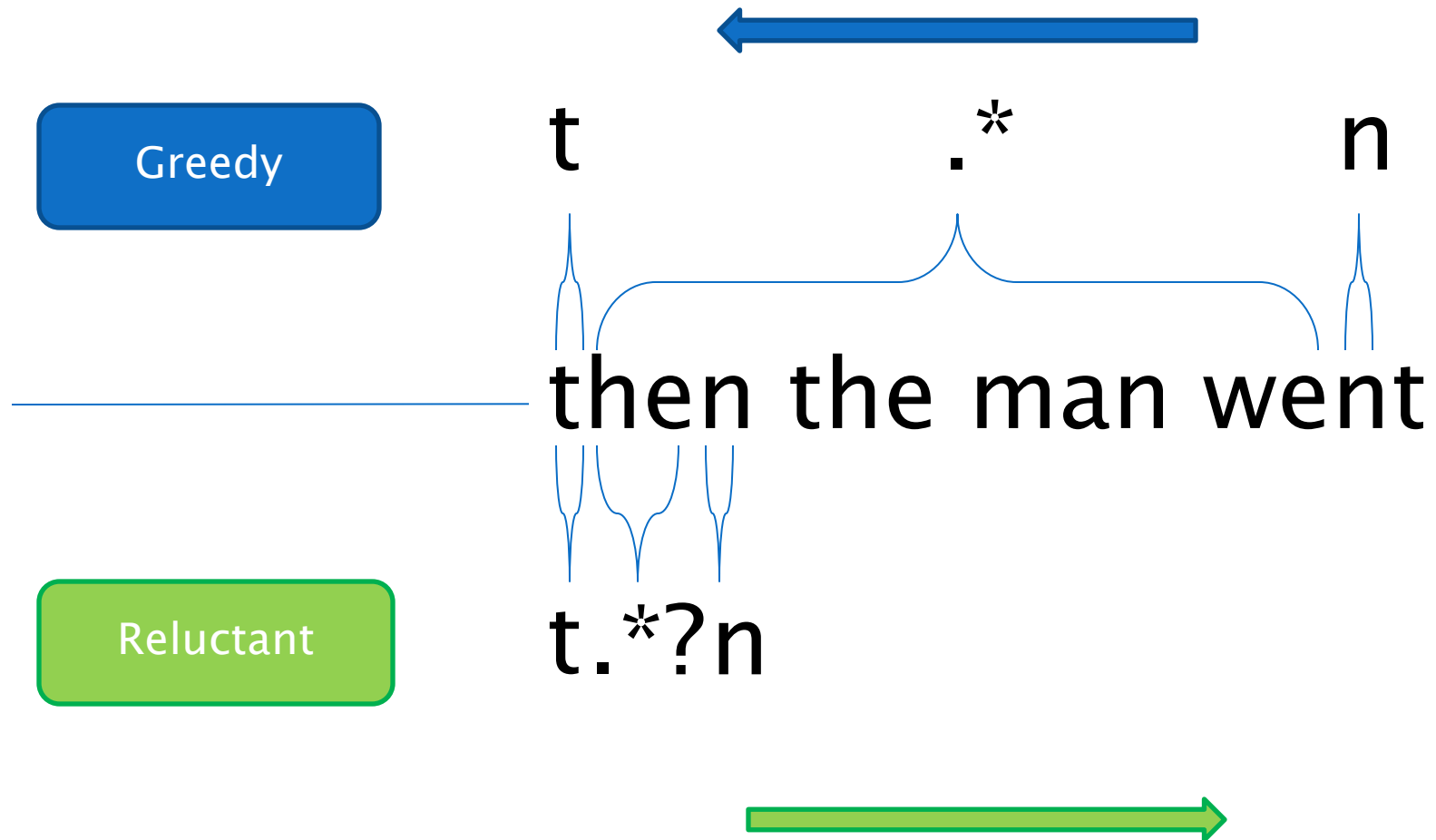
- Take the input `then man`
- The `t.*n` pattern (greedy) gives...

<code>group()</code>	<code>start()</code>	<code>end()</code>
<code>then man</code>	0	8

- But the `t.*?n` pattern (reluctant) gives...

<code>group()</code>	<code>start()</code>	<code>end()</code>
<code>then</code>	0	4

Explained...



Example: Reluctant vs. Greedy II

- Take the input `axxa axxa axxa`

- The `a.*a` pattern (greedy) gives...

<code>group()</code>	<code>start()</code>	<code>end()</code>
<code>axxa axxa axxa</code>	0	14

- But the `a.*?a` pattern (reluctant) gives...

<code>group()</code>	<code>start()</code>	<code>end()</code>
<code>axxa</code>	0	4
<code>axxa</code>	5	9
<code>axxa</code>	10	14

Groups

- We've already seen that we can define groups by using parenthesis
- The matcher can provide the exact substring that each group matched
 - `groupCount` - returns the number of groups
 - `group(int num)` – returns the substring matched for the given group number
 - `start(int num)` and `end(int num)` – substring indexes for the given group number

Example: Simple Groups

- Take the input `abcde`
- The `(a)(b)` pattern has 2 groups, so it matches both groups as...

N (group #)	group (N)	start (N)	end (N)
0	ab	0	2
1	a	0	1
2	b	1	2

- Group 0 is the entire matched substring

Example: Simple Groups II

- Take the input `abab`
- The `(a)(b)` pattern has 2 groups, so it matches both groups twice as...

match #	N (group #)	group (N)	start (N)	end (N)
1	0	ab	0	2
	1	a	0	1
	2	b	1	2
2	0	ab	2	4
	1	a	2	3
	2	b	3	4

Backreferences

- A powerful feature of groups is that you can reference a group elsewhere in the pattern
- The groups are referenced as `\1`, `\2`, `\3` etc
- For example: `(\d\d) \1`
 - The group `(\d\d)` will match any two digits
 - The backreference `\1` means that the value matched by that group must occur again
 - So it will match 1212, 8787, 6464 etc



Boundary Meta-characters

- So far we haven't been concerned where in the input that a match occurs
 - looking for `ab` in `ab cab ab` matches in 3 different places
- We can use boundary meta-characters to specify locations in the input, e.g.
 - `\A` – matches the start of the input
 - `\Aab` will only match once in `ab cab abc`

Boundary Meta-characters

Character	Will match...
\A	the start of the input
\Z	the end of the input before any terminators (carriage returns etc)
\z	the end of the input
^	the start of a line
\$	the end of a line
\b	a word boundary
\B	a non-word boundary
\G	end of the previous match

Exercises

1. Write a Java program to validate IP address using regular expression.
2. Write a Java program to validate email address using regular expression.
3. Write a Java program to validate Rwandan mobile telephone number using regular expression. The program should display message “Local number format” in case a number entered is starting with 07 or message “International number format” in case the entered number is starting with +2507
4. Write a Java program to validate strong password using regular expression. A strong password should contain at least 8 characters including alphanumeric characters, special characters and at least one character should be capital character.
5. Give example inputs that match the following patterns:
 - a. `(0?[1-9]|[12][0-9]|3[01])/(0?[1-9]|1[012])/((19|20)\d\d)`
 - b. `([^\s]+(\.(?i)(txt|doc|csv|pdf)))$`
6. Consider the regular expression `(dog){3}`. Identify the two subexpressions. What string does the expression match?



Reference

- <https://www.oracle.com/ocom/groups/public/@otn/documents/webcontent/272158.htm>
- <https://www.digitalocean.com/community/tutorials/regular-expression-in-java-regex-example>

EoF

